

Hardware-Praktikum 2026

O. Amft und L. Häusler, Intelligent Embedded Systems Lab,
Institute for Computer Science, Albert-Ludwigs-University Freiburg, Freiburg im Breisgau

Exercise sheet 4 (25 + 3) Points

Objective

In this exercise, you will develop a wearable system using an integrated IMU (Inertial Measurement Unit) sensor to recognize arm orientation, dynamic motion, and simple gestures.

Unlike previous exercises, you will work on different modules within a single integrated program/script. The goals are to:

- Read and interpret IMU sensor data (accelerometer and gyroscope).
- Implement orientation detection using static acceleration data.
- Implement gesture detection using temporal acceleration patterns.
- Detect dynamic motion (rotation and directional movement) using gyroscope data.
- Transmit sensor data wirelessly via Bluetooth Low Energy (BLE).
- Visualize the results on a host system in real time.

Implementation Notes

- All sections marked with TODO in the provided `ex04.ino` file must be implemented.
- The visualization component (Part F) must be implemented in the provided `plot.py` script.
- **Important Note:** Please ensure you use the *nRF52 mbed-enabled board*. Using a standard nRF52 board may result in compatibility errors with the BLE (Bluetooth Low Energy) functionality.

System Overview

Hardware Setup

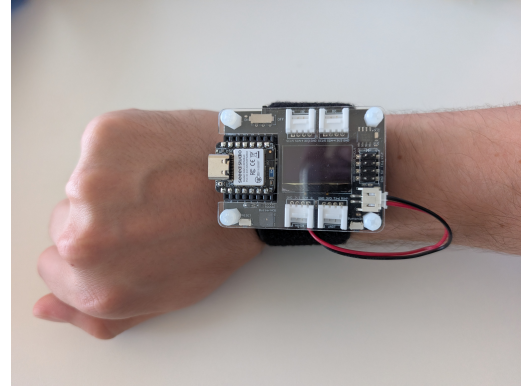
- XIAO nRF52840 Sense
- Integrated IMU (LSM6DS3)
- Wrist-mount (Optional)

As shown in Figure 1, there are two ways to set up your hardware. The only required part is the XIAO nRF52840 Sense microcontroller. You may keep the device connected to your computer via USB and hold it in your hand to complete all tasks in this and the subsequent exercise sheet.

Alternatively, you may borrow the custom wrist-mount and battery to transform your microcontroller into a wearable device. Further information on how to borrow this hardware will be provided on Monday, June 22nd.



(a) Option 1: Xiao microcontroller connected via USB and held by hand.



(b) Option 2: Xiao and expansion board mounted on the 3D-printed wristband with battery.

Abbildung 1: Possible hardware configurations for this exercise.

Part A: IMU Data Acquisition (2 Points)

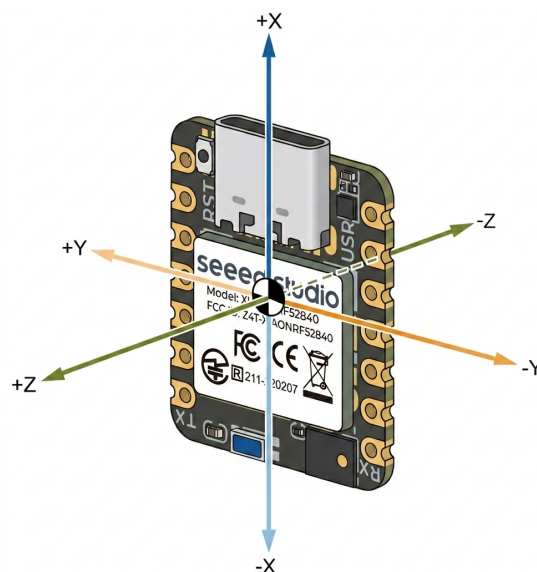
Task

Install the **LSM6DS3 Arduino library** by following the instructions here:

<https://wiki.seeedstudio.com/XIAO-BLE-Sense-IMU-Usage/>

Implement a routine to read the raw acceleration data (a_x, a_y, a_z) and output the values via the Serial Monitor.

Understanding the Axes



An IMU combines an accelerometer (measuring linear acceleration) and a gyroscope (measuring angular velocity). Even when stationary, the accelerometer detects gravity ($\approx 9.81 \text{ m/s}^2$), which

allows the system to determine the device's orientation relative to the Earth. The acceleration axes are defined as follows:

- a_x : Forward / Backward motion
- a_y : Left / Right tilt
- a_z : Vertical motion (Gravity)

Part B: Orientation (3 Points)

Task

Detect and print the orientation of the device.



Requirements

- Use a_z for:
 - Face up
 - Face down
- Use a_y for:
 - Left
 - Right

If no clear orientation can be detected, print "UNKNOWN".

Hint

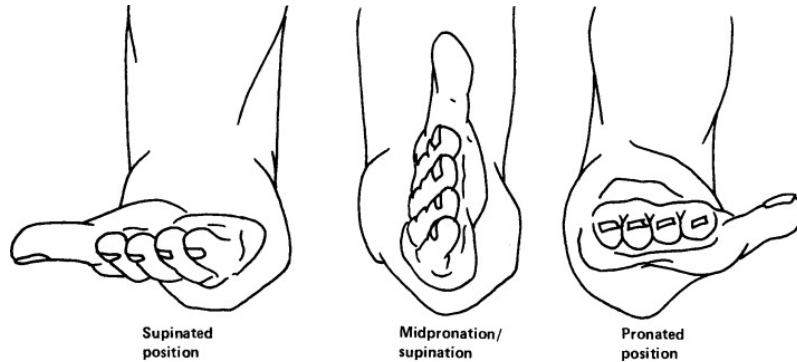
Gravity $\approx 9.81 \text{ m/s}^2$ can be used as a reference.

Part C: Gesture using accelerometer (5 Points)

Task

Detect:

- Supination (face up → face down)
- Pronation (face down → face up)
- None, if no gesture was detected



Requirements

- Use multiple samples (time window)
- Do not rely on a single measurement

Hint

- Store last N values of a_z
- Compute min and max values
- Large difference → movement
- Compare first and last value for direction

Part D: Dynamic Gesture Detection using Gyroscope (5 Points)

Objective

Extend your system by using gyroscope data to detect dynamic arm movements.

Background

While the accelerometer is useful for detecting static orientation (e.g., face up/down), the gyroscope measures **angular velocity** (in degrees per second) and is better suited for detecting **dynamic motion** such as tilting and directional movements.

Task

- Read gyroscope values ($gyrX$, $gyrY$, $gyrZ$)
- Detect directional movements based on angular velocity
- Classify the strongest motion direction

Gesture Definitions

Assume the resting position is:

- Wrist facing upwards
- Accelerometer approximately: $ax \approx 0$, $ay \approx 0$, $az \approx 9.8$

Based on gyroscope data:

- **X-axis:** Tilt Left / Tilt Right
- **Y-axis:** Move Up / Move Down
- **Z-axis:** Move Left / Move Right

Hints

- Use a threshold (e.g., ≈ 100 deg/s) to detect significant motion
- Compare absolute values using `abs()` to find the strongest movement
- Only return the dominant motion direction (highest magnitude)

Expected Output

Examples of detected gestures:

TILT_LEFT
TILT_RIGHT
MOVE_UP
MOVE_DOWN
MOVE_LEFT
MOVE_RIGHT
NONE

Part E: Bluetooth Low Energy (BLE) Communication (5 Points)

Objective

Extend your system by transmitting IMU data wirelessly using Bluetooth Low Energy (BLE).

Task

- Initialize BLE on the XIAO nRF52840 Sense
- Create a BLE service and characteristic
- Continuously send IMU data via BLE (same format as Serial output)

Testing with nRF Connect

To verify your BLE implementation, use a smartphone application such as **nRF Connect**.

1. Install the app (Android/iOS)
2. Enable Bluetooth on your phone
3. Start scanning for devices
4. Look for your device (e.g., XIAO-IMU)
5. Connect to the device
6. Locate the BLE characteristic (UUID defined in your code)
7. Enable **notifications**

Expected Result

After enabling notifications, you should see a continuous stream of data similar to:

ax: -0.05 | ay: -0.16 | az: 9.93 | gyrX: 0.21 | gyrY: -1.54 | gyrZ: -0.35
Orientation: FACE UP | FSM Gesture: SUPINATION | Gyro Gesture: TILT_RIGHT

Hints

- Use `BLE.begin()` to initialize BLE
- Use `BLE.setLocalName()` to name your device
- Use `BLENotify` to enable data streaming
- Convert Arduino String to `char*` using `.c_str()`

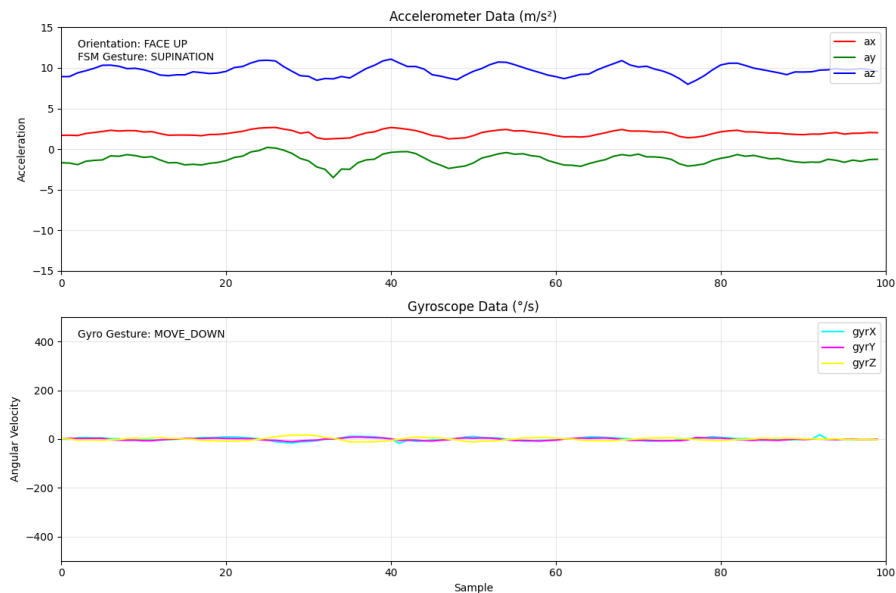
Troubleshooting

- If the device is not visible: check if BLE advertising is started
- If no data appears: ensure notifications are enabled
- If connection fails: verify UUIDs match between code and client

Part F: Visualization (5 Points)

Task

Visualize the sensor data and detection results on your computer in real time. Implement the visualization logic in the provided `plot.py` file.



Requirements

- Plot the raw IMU data (acceleration/gyroscope) as a function of time.
- Display the currently detected orientation and gesture (e.g., as text overlays or labels on the plot).

Environment Setup

You can use the provided setup files to create a dedicated Python virtual environment and install the required dependencies from `requirements.txt`. Run `setup.bat` on Windows or `setup.sh` on Linux/macOS to automatically configure your environment.

Important Note

Warning: Close the Arduino Serial Monitor before running the Python script.

Bonus: Robust Gesture Detection (State Machine) (3 Points)

Objective

Improve the robustness of your gesture detection by introducing a state machine.

Task

Instead of directly detecting gestures from the IMU data, implement a finite state machine (FSM) that models the temporal behavior of the system.

Requirements

- Define at least three states:
 - IDLE (no motion)
 - MOVING (motion detected)
 - DETECTED (gesture recognized)
- Implement transitions between states based on IMU data
- Ensure that gestures are only detected in valid transitions

Hint

- Use the range of acceleration values to detect movement
- Only classify gestures when sufficient motion is detected
- Reset the system to IDLE after a gesture is detected

Evaluation Criteria

- Correct data acquisition
- Functional orientation detection
- Robust gesture detection
- Working visualization

Submission

- Complete the provided `ex04.ino` and `plot.py` files.
- Verify that your system works correctly (IMU reading, gesture detection, and visualization).
- Provide a screenshot from the **nRF Connect** app showing the transmitted BLE data.
- Submit all files in a single `.zip` archive.